

系统开发工具基础

第三周实验报告

姓名：颜煜

学号：2502007152

1 课堂实验内容与结果

1.1 实验九：Python 包构建与分发包生成

```
yy@yy:~$ # 1. 创建项目目录结构
mkdir -p q09/src/greetlab
cd q09
yy@yy:~/q09$ touch src/greetlab/__init__.py
yy@yy:~/q09$ cat > src/greetlab/cli.py << 'EOF'
import argparse
def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print(f"Hello, {a.name}!")
EOF
yy@yy:~/q09$ cat > pyproject.toml << 'EOF'

[build-system]

requires = ["setuptools>=68"]

build-backend = "setuptools.build_meta"

[project]

name = "greetlab-25020007152"

version = "0.1.0"

[project.scripts]

sdt-greet = "greetlab.cli:main"

EOF

yy@yy:~/q09$ python3 -m pip install build
/usr/bin/python3: No module named pip
yy@yy:~/q09$ sudo apt update
[sudo] password for yy:
命中:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
命中:2 https://packages.microsoft.com/repos/code stable InRelease
命中:3 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy InRelease
命中:4 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates InRelease
命中:5 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-backports InRelease
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
有 169 个软件包可以升级。请执行 'apt list --upgradable' 来查看它们。
yy@yy:~/q09$ sudo apt install -y python3-pip
```

图 1: Enter Caption

```

yy@yy:~/q09$ python3 -m pip install --upgrade --user setuptools -i https://pypi.tuna.tsinghua.edu.cn/simple
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Requirement already satisfied: setuptools in /usr/lib/python3/dist-packages (59.6.0)
Collecting setuptools
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/95/9c/c510029fc6ef33a6275cd2c5d3cecd6613dfd6aa401d57c54f1c18852ccf/setuptools-84.0.0-py3-none-any.whl (818 kB)
    818.2/818.2 KB 574.6 kB/s eta 0:00:00
Installing collected packages: setuptools
Successfully installed setuptools-84.0.0
yy@yy:~/q09$ python3 -c "import setuptools; print(setuptools.__version__)"
84.0.0
yy@yy:~/q09$ python3 -m build --no-isolation

```

图 2: Enter Caption

```

Successfully built greetlab_25020007152-0.1.0.tar.gz and greetlab_25020007152-0.1.0-py3-none-any.whl
yy@yy:~/q09$ ls dist/
greetlab_25020007152-0.1.0-py3-none-any.whl  greetlab_25020007152-0.1.0.tar.gz
yy@yy:~/q09$ cd ..
yy@yy:~$ python3 -m venv test_wheel_env

yy@yy:~$
yy@yy:~$ source test_wheel_env/bin/activate
(test_wheel_env) yy@yy:~$ ls q09/dist/
greetlab_25020007152-0.1.0-py3-none-any.whl  greetlab_25020007152-0.1.0.tar.gz
(test_wheel_env) yy@yy:~$ ^C
(test_wheel_env) yy@yy:~$ pip install q09/dist/greetlab_25020007152-0.1.0-py3-none-any.whl
Processing ./q09/dist/greetlab_25020007152-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-25020007152
Successfully installed greetlab-25020007152-0.1.0
(test_wheel_env) yy@yy:~$ sdt-greet --name 25020007152
Hello, 25020007152!

```

图 3: Enter Caption

1.1.1 实验目的

掌握现代 Python 包的标准目录结构与配置规范，理解 pyproject.toml 配置文件的作用，学会使用 build 工具构建源码分发包（sdist）与 wheel 二进制包，掌握本地包安装与命令行功能验证的方法。

1.1.2 操作步骤

1. 创建包目录结构与代码文件创建 q09 实验目录,按照 Python 包规范建立 src/greetlab 目录，编写模块文件与命令行入口。

```

mkdir -p q09/src/greetlab
cd q09
cat > src/greetlab/__init__.py << 'EOF'

```

```

__version__ = "0.1.0"
EOF
cat > src/greetlab/cli.py << 'EOF'
import argparse

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--name", required=True)
    args = parser.parse_args()
    print(f"Hello, {args.name}!")
EOF

```

2. 编写打包配置文件创建 `pyproject.toml`，声明构建后端、包元数据与命令行入口点。

```

cat > pyproject.toml << 'EOF'
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab"
version = "0.1.0"

[project.scripts]
sdt-greet = "greetlab.cli:main"
EOF

```

3. 安装构建工具并执行构建安装 `build` 工具，使用非隔离模式执行构建，规避网络超时间问题。

```

python3 -m pip install --user build -i https://pypi.tuna.tsinghua.edu.cn/simple
python3 -m build --no-isolation

```

4. 验证构建产物与功能查看生成的分发包，创建测试虚拟环境安装 `wheel` 包，验证命令行功能。

```

ls dist/
python3 -m venv test_wheel_env

```

```
source test_wheel_env/bin/activate
pip install dist/greetlab-0.1.0-py3-none-any.whl
sdt-greet --name test
EOF
```

1.1.3 实验结果

1. **包结构规范有效**: 按照 src 布局的包结构符合现代 Python 打包规范, 模块导入与命令入口配置正确。
2. **构建产物完整生成**: 成功在 dist 目录生成源码包 greetlab-0.1.0.tar.gz 与 wheel 包 greetlab-0.1.0-py3-none-any.whl。
3. **安装后功能正常**: 虚拟环境中安装 wheel 包后, sdt-greet 命令可正常调用, 输入姓名后输出对应问候语。
4. **非隔离构建可行**: 使用 `-no-isolation` 跳过隔离构建环境, 成功规避网络超时问题, 构建结果与标准模式一致。

1.1.4 问题与解决

- **问题**: 标准构建模式下下载依赖超时, 构建失败。
- **原因**: 隔离构建环境需要联网从 PyPI 下载依赖, 境外服务器连接不稳定。
- **解决**: 添加 `-no-isolation` 参数使用本地已有依赖构建, 无需联网下载, 构建顺利完成。
- **问题**: 系统 `setuptools` 版本过低, 不满足构建配置要求。
- **原因**: 系统自带 `setuptools` 为 59.6 版本, 配置要求大于等于 68。
- **解决**: 使用国内镜像升级 `setuptools`, 或根据实际版本调整 `pyproject.toml` 中的版本要求。

1.1.5 实验小结

现代 Python 包以 `pyproject.toml` 作为统一配置文件, 替代了传统的 `setup.py`。src 目录布局可以有效避免导入污染, 是推荐的包结构。sdist 是源码分发包, 兼容性最强; wheel 是预构建的二进制分发包, 安装速度更快。构建时的隔离环境可以保证构建环境纯净, 但在网络受限场景下, 非隔离模式是可行的替代方案。

1.2 实验十：测试驱动的输出校验修复

```
yy@yy:~/q10$ cd ~
cp -r q09 q10
cd q10
yy@yy:~/q10$ python3 -m pip install --user pytest -i https://pypi.tuna.tsinghua.edu.cn/simple
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Requirement already satisfied: pytest in /usr/lib/python3/dist-packages (6.2.5)
4yy@yy:~/q10$ cat > test_cli.py << 'EOF'
import pytest
from greetlab.cli import main
import sys

def test_normal_name(capsys):
    # 正常名称测试
    sys.argv = ["sdt-greet", "--name", "test"]
    main()
    captured = capsys.readouterr()
    assert "Hello, test!" in captured.out

def test_blank_name_exit_code():
    # 全空白名称测试：应抛出SystemExit且退出码为2
    sys.argv = ["sdt-greet", "--name", " "]
    with pytest.raises(SystemExit) as exc_info:
        main()
    assert exc_info.value.code == 2
EOF
yy@yy:~/q10$ python3 -m pytest test_cli.py -v
===== test session starts =====
platform linux -- Python 3.10.12, pytest-6.2.5, py-1.10.0, pluggy-0.13.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/yy/q10
collected 0 items / 1 error

===== ERRORS =====
_____ ERROR collecting test_cli.py _____
ImportError while importing test module '/home/yy/q10/test_cli.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
/usr/lib/python3.10/importlib/__init__.py:126: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
test_cli.py:2: in <module>
    from greetlab.cli import main
E   ModuleNotFoundError: No module named 'greetlab'
===== short test summary info =====
ERROR test_cli.py
!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!!
===== 1 error in 0.05s =====
yy@yy:~/q10$ python3 -m pip install -e . --user
Obtaining file:///home/yy/q10
Installing build dependencies ... done
```

图 4: Enter Caption

```

File Edit View Search Terminal Help
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
ERROR: Project file:///home/yy/q10 has a 'pyproject.toml' and its build backend is missing the 'build_editable' hook. Since it does
not have a 'setup.py' nor a 'setup.cfg', it cannot be installed in editable mode. Consider using a build backend that supports PEP
660.
yy@yy:~/q10$ PYTHONPATH=src python3 -m pytest test_cli.py -v
===== test session starts =====
platform linux -- Python 3.10.12, pytest-6.2.5, py-1.10.0, pluggy-0.13.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/yy/q10
collected 2 items

test_cli.py::test_normal_name PASSED [ 50%]
test_cli.py::test_blank_name_exit_code FAILED [100%]

===== FAILURES =====
_____ test_blank_name_exit_code _____

    def test_blank_name_exit_code():
        # 全空白名称测试: 应抛出SystemExit且退出码为2
        sys.argv = ["sdt-greet", "--name", " "]
>       with pytest.raises(SystemExit) as exc_info:
E       Failed: DID NOT RAISE <class 'SystemExit'>

test_cli.py:15: Failed
----- Captured stdout call -----
Hello,  !

===== short test summary info =====
FAILED test_cli.py::test_blank_name_exit_code - Failed: DID NOT RAISE <class 'SystemExit'>
===== 1 failed, 1 passed in 0.06s =====
yy@yy:~/q10$ cat > src/greetlab/cli.py << 'EOF'
import argparse
import sys

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()

    # name 去除首尾空白后为空则退出, 退出码为 2
    if not a.name.strip():
        sys.exit(2)

    print(f"Hello, {a.name}!")
EOF
yy@yy:~/q10$ PYTHONPATH=src python3 -m pytest test_cli.py -v
===== test session starts =====
platform linux -- Python 3.10.12, pytest-6.2.5, py-1.10.0, pluggy-0.13.0 -- /usr/bin/python3

```

图 5: Enter Caption

```

cachedir: .pytest_cache
rootdir: /home/yy/q10
collected 2 items

test_cli.py::test_normal_name PASSED [ 50%]
test_cli.py::test_blank_name_exit_code PASSED [100%]

===== 2 passed in 0.01s =====
yy@yy:~/q10$ cat > ai_log.md << 'EOF'
1. 核心提示：当--name参数仅含空白字符时，程序需以SystemExit(2)退出
2. 智能体改动：在cli.py中添加name.strip()空值判断，命中则sys.exit(2)
3. 人工验证：运行pytest测试，正常名称输出正确，空白名称正确返回退出码2
4. 无无关修改，仅新增空白校验逻辑，原有功能不受影响
EOF

```

图 6: Enter Caption

1.2.1 实验目的

理解测试驱动开发的核心流程，掌握 pytest 单元测试的编写方法，学会通过测试验证代码修复效果，掌握程序退出码的测试方式，理解输入合法性校验的重要性。

1.2.2 操作步骤

1. 复制项目并添加测试用例复制实验九的项目为 q10，编写测试文件覆盖正常场景与空白输入异常场景。

```

cp -r q09 q10
cd q10
cat > test_cli.py << 'EOF'
import pytest
import sys
from greetlab.cli import main

def test_normal_name(capsys):
    sys.argv = ["sdt-greet", "--name", "test"]
    main()
    captured = capsys.readouterr()
    assert "Hello, test!" in captured.out

def test_blank_name_exit_code():
    sys.argv = ["sdt-greet", "--name", " "]
    with pytest.raises(SystemExit) as exc_info:

```

```
        main()
    assert exc_info.value.code == 2
EOF
```

2. 运行测试确认失败状态通过环境变量指定模块路径，运行测试验证初始代码未做空白校验，测试失败。

```
PYTHONPATH=src python3 -m pytest test_cli.py -v
```

3. 修改代码实现空白校验在命令行入口函数中添加空白字符判断，全空白输入时以状态码 2 退出。

```
cat > src/greetlab/cli.py << 'EOF'
import argparse
import sys

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--name", required=True)
    args = parser.parse_args()

    if not args.name.strip():
        sys.exit(2)

    print(f"Hello, {args.name}!")
EOF
```

4. 重新运行测试并记录过程再次运行测试验证修复效果，创建 ai_log.md 记录修复全过程。

```
PYTHONPATH=src python3 -m pytest test_cli.py -v
cat > ai_log.md << 'EOF'
```

1. 核心提示：当--name参数仅含空白字符时，程序需以SystemExit(2)退出。
2. 改动：在cli.py中添加name.strip()空值判断，命中则sys.exit(2)。
3. 验证：运行pytest测试，正常名称输出正确，空白名称返回退出码2。
4. 无无关修改，仅新增空白校验逻辑，原有功能不受影响。

```
EOF
```

1.2.3 实验结果

1. **初始测试准确失败**：未修复前，空白名称测试用例失败，程序正常输出问候语并以 0 退出，符合缺陷预期。
2. **修复后测试全部通过**：添加空白校验后，两个测试用例全部通过，正常功能保留，异常输入正确触发退出。
3. **退出码符合要求**：空白输入时程序准确以状态码 2 终止，可被调用方通过退出码识别非法输入。
4. **过程记录完整规范**：ai_log.md 清晰记录了修复目标、代码改动与验证结果，符合可追溯要求。

1.2.4 问题与解决

- **问题**：测试文件无法导入 greetlab 模块，报 ModuleNotFoundError。
- **原因**：包位于 src 子目录，Python 默认搜索路径不包含该目录。
- **解决**：运行测试时通过 PYTHONPATH=src 临时添加模块搜索路径，无需安装即可测试本地源码。
- **问题**：可编辑模式安装失败，不支持 build_editable。
- **原因**：低版本 setuptools 不支持基于 pyproject.toml 的可编辑安装。
- **解决**：使用环境变量方式替代可编辑安装，修改源码即时生效，满足测试驱动开发需求。

1.2.5 实验小结

测试驱动开发遵循“先写失败测试，再实现功能，最后测试验证”的循环，可以保证代码完全符合需求。pytest 的 capsys 可以捕获标准输出，pytest.raises 可以捕获异常并验证退出码，是测试命令行程的标准手段。输入合法性校验是程序健壮性的基础，对空白、非法格式等异常输入做明确处理，可以避免下游流程出现不可预期的错误。

1.3 实验十一：技术协作材料规范化改写

```
yy@yy:~/q10$ cd ..
yy@yy:~$ mkdir -p ~/q11
cd ~/q11
yy@yy:~/q11$ cat > communication.md << 'EOF'
# 协作材料改写

## Issue：空白名称参数仍输出无效问候且正常退出
- 环境：操作系统版本待确认，Python 版本待确认，greetlab 版本 0.1.0
- 复现命令：`sdt-greet --name " "`
- 期望结果：程序识别非法空白输入，以退出码 2 终止，不输出问候语
- 实际结果：程序正常执行并以退出码 0 退出，输出 `Hello,  !`

## 提交信息
**标题：修复空白名称参数未校验的问题**
正文：传入仅含空白字符的 `--name` 参数时，程序未执行合法性校验，直接输出无效问候并以 0 正常退出，不符合输入校验规范。本次修改在 main 函数中新增空白字符判断，去除首尾空白后为空则调用 `sys.exit(2)` 终止程序，确保非法输入可被调用方通过退出码识别。

## 评审意见
**[Blocking] 空白输入未做合法性校验**
- 具体行为：当前实现未对 `--name` 参数进行空白字符过滤，全空白名称可正常通过解析并输出结果。
- 风险：产生无意义的无效输出，调用方无法通过退出码区分合法与非法输入，下游依赖退出状态的流程会出现异常。
- 建议动作：补充空白字符校验逻辑，全空白输入时返回非零退出码；同步新增对应单元测试覆盖该场景，验证修复效果。
EOF
yy@yy:~/q11$ cat communication.md
# 协作材料改写

## Issue：空白名称参数仍输出无效问候且正常退出
- 环境：操作系统版本待确认，Python 版本待确认，greetlab 版本 0.1.0
- 复现命令：`sdt-greet --name " "`
- 期望结果：程序识别非法空白输入，以退出码 2 终止，不输出问候语
- 实际结果：程序正常执行并以退出码 0 退出，输出 `Hello,  !`

## 提交信息
**标题：修复空白名称参数未校验的问题**
正文：传入仅含空白字符的 `--name` 参数时，程序未执行合法性校验，直接输出无效问候并以 0 正常退出，不符合输入校验规范。本次修改在 main 函数中新增空白字符判断，去除首尾空白后为空则调用 `sys.exit(2)` 终止程序，确保非法输入可被调用方通过退出码识别。

## 评审意见
**[Blocking] 空白输入未做合法性校验**
- 具体行为：当前实现未对 `--name` 参数进行空白字符过滤，全空白名称可正常通过解析并输出结果。
- 风险：产生无意义的无效输出，调用方无法通过退出码区分合法与非法输入，下游依赖退出状态的流程会出现异常。
- 建议动作：补充空白字符校验逻辑，全空白输入时返回非零退出码；同步新增对应单元测试覆盖该场景，验证修复效果。
```

图 7: Enter Caption



图 8: Enter Caption

1.3.1 实验目的

掌握技术协作中 Issue、提交信息、评审意见的规范写作方法，理解可复现、可执行信息在团队协作中的价值，学会标注问题严重级别，提升技术沟通的效率与准确性。

1.3.2 操作步骤

创建 q11 实验目录，编写 communication.md，分别重写三份协作材料。

```
mkdir -p q11
cd q11
cat > communication.md << 'EOF'
# 协作材料改写
```

Issue：空白名称参数仍输出无效问候且正常退出

- 环境：操作系统版本待确认，Python版本待确认，greetlab版本0.1.0
- 复现命令：`sdt-greet --name " "`
- 期望结果：程序识别非法空白输入，以退出码2终止，不输出问候语
- 实际结果：程序正常执行并以退出码0退出，输出Hello, !

提交信息

****标题：修复空白名称参数未校验的问题****

正文：传入仅含空白字符的`--name`参数时，程序未执行合法性校验，直接输出无效问候并以0正常退出

评审意见

****[Blocking] 空白输入未做合法性校验****

- 具体行为：当前实现未对`--name`参数进行空白字符过滤，全空白名称可正常通过解析并输出结果。
 - 风险：产生无意义的无效输出，调用方无法通过退出码区分合法与非法输入，下游依赖退出状态的
 - 建议动作：补充空白字符校验逻辑，全空白输入时返回非零退出码；同步新增对应单元测试覆盖该
- EOF

1.3.3 实验结果

1. **Issue 信息完整可复现**：包含环境、复现命令、期望结果与实际结果，未知信息明确标注待确认，接收方可以直接复现问题。

2. **提交信息清晰明确**：标题使用祈使语气，正文准确说明问题原因与解决方案，审阅者可以快速理解修改意图。
3. **评审意见专业可执行**：明确指出具体问题、带来的风险与建议动作，标注 Blocking 严重级别，优先级清晰。
4. **篇幅符合要求**：全文表述简洁专业，无冗余信息。

1.3.4 问题与解决

- **问题**：原始协作材料信息模糊，无法直接执行。
- **原因**：原始描述过于简略，缺少复现步骤、预期行为等关键信息，接收方无法准确理解需求。
- **解决**：按照规范补充完整上下文，明确操作命令与预期结果，将模糊描述转化为可执行信息。

1.3.5 实验小结

高效的技术协作依赖标准化的信息表达。Issue 的核心是可复现性，必须提供完整的环境、操作步骤与预期；提交信息要准确传达修改目的与内容，方便代码审阅与版本回溯；评审意见需要对事不对人，明确指出问题、影响与改进方向，并标注严重级别。规范的协作材料可以大幅减少沟通成本，提升团队开发效率。

1.4 实验十二：PyTorch 线性回归训练循环实现

```
yy@yy:~$ mkdir -p ~/q12
cd ~/q12
yy@yy:~/q12$ cat > train.py << 'EOF'
import torch
from torch import nn

torch.manual_seed(20260907)
x = torch.linspace(-1, 1, 101).reshape(-1, 1)
y = 3 * x - 1

model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)

for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    # 补全：清空梯度、反向传播、更新参数
    opt.zero_grad()
    loss.backward()
    opt.step()

# 补全：进入评估模式，无梯度环境下计算并打印结果
model.eval()
with torch.no_grad():
    final_pred = model(x)
    final_loss = loss_fn(final_pred, y)
    print(f"最终损失: {final_loss.item():.6f}")
    print(f"weight: {model.weight.item():.6f}")
    print(f"bias: {model.bias.item():.6f}")
EOF
```

图 9: Enter Caption

```

yy@yy:~$ python3 -m pip install --user torch --index-url https://download.pytorch.org/whl/cpu
Looking in indexes: https://download.pytorch.org/whl/cpu
Collecting torch
  Downloading https://download-r2.pytorch.org/whl/cpu/torch-2.14.0%2Bcpu-cp310-cp310-manylinux_2_28_x86_64.whl (196.2 MB)
    196.2/196.2 MB 7.5 MB/s eta 0:00:00
Requirement already satisfied: typing-extensions>=4.10.0 in ~/.local/lib/python3.10/site-packages (from torch) (4.16.0)
Requirement already satisfied: fsspec>=0.8.5 in ~/.local/lib/python3.10/site-packages (from torch) (2026.7.0)
Requirement already satisfied: sympy>=1.13.3 in ~/.local/lib/python3.10/site-packages (from torch) (1.14.0)
Requirement already satisfied: Jinja2 in ~/.local/lib/python3.10/site-packages (from torch) (3.1.6)
Requirement already satisfied: networkx>=2.5.1 in ~/.local/lib/python3.10/site-packages (from torch) (3.4.2)
Requirement already satisfied: setuptools>=77.0.3 in ~/.local/lib/python3.10/site-packages (from torch) (84.0.0)
Requirement already satisfied: filelock in ~/.local/lib/python3.10/site-packages (from torch) (3.32.6)
Requirement already satisfied: mpmath<1.4, >=1.1.0 in ~/.local/lib/python3.10/site-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/lib/python3/dist-packages (from Jinja2->torch) (2.0.1)
Installing collected packages: torch
Successfully installed torch-2.14.0+cpu
yy@yy:~$ python3 train.py
python3: can't open file '/home/yy/train.py': [Errno 2] No such file or directory
yy@yy:~$ cd ~/q12
yy@yy:~/q12$ python3 train.py
/home/yy/.local/lib/python3.10/site-packages/torch/_subclasses/functional_tensor.py:368: UserWarning: Failed to initialize NumPy: N
o module named 'numpy' (Triggered internally at /__w/pytorch/pytorch/torch/csrc/utils/tensor_numpy.cpp:84.)
  cpu = _conversion_method_template(device=torch.device("cpu"))
最终损失: 0.000000
weight: 2.999997
bias: -1.000000

```

图 10: Enter Caption

1.4.1 实验目的

掌握 PyTorch 张量操作与线性层的使用，理解深度学习训练循环的标准流程，包括梯度清零、反向传播与参数更新，学会评估模式与无梯度推理的使用方法，验证梯度下降在线性回归中的收敛效果。

1.4.2 操作步骤

1. 创建实验目录与训练代码创建 q12 目录，编写训练脚本，补全训练循环与评估阶段的 TODO 部分。

```

mkdir -p q12
cd q12
cat > train.py << 'EOF'
import torch
from torch import nn

torch.manual_seed(20260907)
x = torch.linspace(-1, 1, 101).reshape(-1, 1)
y = 3 * x - 1

```

```

model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)

for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    opt.zero_grad()
    loss.backward()
    opt.step()

model.eval()
with torch.no_grad():
    final_pred = model(x)
    final_loss = loss_fn(final_pred, y)
    print(f"最终损失: {final_loss.item():.6f}")
    print(f"weight: {model.weight.item():.6f}")
    print(f"bias: {model.bias.item():.6f}")
EOF

```

2. 安装 PyTorch 并运行训练

安装 CPU 版本 PyTorch，执行训练脚本。

```

python3 -m pip install --user torch --index-url https://download.pytorch.org/whl/cpu
python3 train.py

```

1.4.3 实验结果

1. **训练循环逻辑正确**：完整实现梯度清零、反向传播、参数更新的标准三步流程，训练过程稳定收敛。
2. **损失收敛符合预期**：200 轮训练后最终损失小于 0.001，模型拟合程度很高。
3. **参数准确收敛**：训练后 weight 接近 3，bias 接近-1，与目标函数 $y = 3 * x - 1$ 的真实参数高度吻合。
4. **评估模式规范正确**：推理阶段切换至评估模式并关闭梯度计算，符合 PyTorch 最佳实践。

1.4.4 问题与解决

- **问题：**磁盘空间不足，无法安装 PyTorch。
- **原因：**虚拟机根分区空间不足，GPU 版本 PyTorch 体积过大。
- **解决：**先扩容虚拟机磁盘与系统分区，再安装纯 CPU 版本 PyTorch，去除冗余的 CUDA 组件，降低安装体积。
- **问题：**安装 PyTorch 时网络下载超时。
- **原因：**默认源下载速度慢，连接不稳定。
- **解决：**使用官方 CPU 专属镜像源安装，下载速度稳定，安装顺利完成。

1.4.5 实验小结

PyTorch 的训练循环遵循固定范式：前向传播计算损失，然后清零梯度，接着反向传播计算梯度，最后由优化器更新参数。`model.eval()` 用于切换评估模式，`torch.no_grad()` 用于关闭梯度计算，二者常用于验证与推理阶段，可以节省内存并提升推理速度。线性回归作为简单的深度学习任务，可以直观展示梯度下降算法的收敛过程，训练后的参数会逐渐逼近真实值。

2 课后

2.1 练习 1: Python 虚拟环境与环境变量对比

```
yy@yy:~$ mkdir -p ex01
cd ex01
yy@yy:~/ex01$ # 保存当前全部环境变量到文件
printenv > before.txt
# 追加记录系统默认 python3 解释器的路径
which python3 >> before.txt
yy@yy:~/ex01$ python3 -m venv myenv
yy@yy:~/ex01$ # 激活虚拟环境
source myenv/bin/activate

# 保存激活后的全部环境变量
printenv > after.txt
# 追加记录虚拟环境内的 python3 路径
which python3 >> after.txt
(myenv) yy@yy:~/ex01$ diff before.txt after.txt
30a31
> VIRTUAL_ENV=/home/yy/ex01/myenv
46a48
> VIRTUAL_ENV_PROMPT=(myenv)
47a50
> PS1=(myenv) \[\e]0;\u@h: \w\a\]\${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@h\[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\
]\$
50c53
< PATH=/home/yy/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap
p/bin
---
> PATH=/home/yy/ex01/myenv/bin:/home/yy/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/l
ocal/games:/snap/bin:/snap/bin
54d56
< _=/usr/bin/printenv
56c58,59
< /usr/bin/python3
---
> _=/usr/bin/printenv
> /home/yy/ex01/myenv/bin/python3
(myenv) yy@yy:~/ex01$ # 退出虚拟环境
deactivate

# 查看退出后的 PATH 变量, 确认恢复为激活前的系统路径
echo $PATH
/home/yy/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin
```

图 11: Enter Caption

实验内容

`mkdir -p ex01`

创建实验目录

`cd ex01`

进入工作目录

```
printenv > before.txt
保存虚拟环境激活前环境变量
which python3 >> before.txt
记录激活前python解释器路径
python3 -m venv myvenv
创建python虚拟环境
source myvenv/bin/activate
激活虚拟环境

printenv > after.txt
保存激活之后的环境变量
which python3 >> after.txt
记录激活后python解释器路径
diff before.txt after.txt
对比激活前后环境差异
deactivate
执行退出虚拟环境操作
echo $PATH
观察退出后的PATH变量状态
```

实验结果

虚拟环境激活仅修改当前shell会话，不会影响其他终端。
激活时主要修改PATH，将虚拟环境bin目录置于搜索路径最前方，系统会优先调用虚拟环境内部的程序，同时新增VIRTUAL_ENV标记。
deactivate是bash shell函数，并非外部程序。
它会读取保存的旧环境变量，把PATH以及相关变量恢复至激活前。
虚拟环境本身文件不会被删除，只是shell不再优先使用该环境。

2.2 练习 2: Python 本地包安装与依赖锁

```
yy@yy:~/ex04$ cd ..
yy@yy:~$ mkdir -p ex02
cd ex02
yy@yy:~/ex02$ mkdir demo_pkg
cat > demo_pkg/__init__.py << 'EOF'
__version__ = "0.1.0"

def hello():
    return "Hello from demo_pkg"
EOF
yy@yy:~/ex02$ cat > pyproject.toml << 'EOF'
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "demo-pkg"
version = "0.1.0"
dependencies = []
EOF
yy@yy:~/ex02$ python3 -m venv env
source env/bin/activate
(env) yy@yy:~/ex02$ pip install .
Processing /home/yy/ex02
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: demo-pkg
  Building wheel for demo-pkg (pyproject.toml) ... done
  Created wheel for demo-pkg: filename=demo_pkg-0.1.0-py3-none-any.whl size=1175 sha256=611c75747de4b7528a8bb9d7f6e7a0c09607cbda456a1fc1585189608a03956d
  Stored in directory: /tmp/pip-ephem-wheel-cache-ctl_z1n3/wheels/fd/fa/56/95238f11c382cdee1e9c3597446b1cf13648d67219b1d4c2a0
Successfully built demo-pkg
Installing collected packages: demo-pkg
Successfully installed demo-pkg-0.1.0
(env) yy@yy:~/ex02$ pip freeze > lock.txt
cat lock.txt
demo-pkg @ file:///home/yy/ex02
(env) yy@yy:~/ex02$ deactivate
```

图 12: Enter Caption

实验内容

`mkdir -p ex02`

创建实验目录

`cd ex02`

进入工作目录

`mkdir demo_pkg`

建立包源码文件夹

`vim demo_pkg/__init__.py`

编写包源码

```
vim pyproject.toml
```

编写现代项目配置文件

```
python3 -m venv env
```

新建隔离虚拟环境

```
source env/bin/activate
```

激活虚拟环境

```
pip install .
```

在虚拟环境安装本地包

```
pip freeze > lock.txt
```

生成依赖锁文件

```
cat lock.txt
```

查看锁文件内容

```
deactivate
```

退出虚拟环境

实验结果 pyproject.toml 是现代 Python 项目标准配置，替代旧的 setup.py。虚拟环境实现依赖隔离，包仅作用于当前 env，不污染系统 Python。pip freeze 输出当前环境全部已经安装的包版本。锁文件固化依赖版本，用于复现完全一致的运行环境。分享项目时带上锁文件，其他人可以复现完全相同依赖。

2.3 练习 3: PyTorch 项目依赖管理

实验内容

```
mkdir -p ex03
```

创建实验目录

```
cd ex03
```

进入工作目录

```
mkdir torch_demo
```

创建包目录

```
vim torch_demo/__init__.py
```

编写简单pytorch示例代码

```
vim pyproject.toml
```

配置项目，声明依赖torch

```
python3 -m venv torch_env
```

创建干净虚拟环境

```
source torch_env/bin/activate
```

激活虚拟环境

```
pip install .
```

安装本项目，自动拉取pytorch依赖

```
pip freeze > torch_lock.txt
```

导出锁文件

```
cat torch_lock.txt
```

查看锁内依赖列表

```
deactivate
```

退出虚拟环境

实验结果 pyproject.toml 的 dependencies 字段声明项目运行依赖。执行 pip install . 的时候会自动下载安装 PyTorch 及其依赖。pip freeze 导出锁文件，完整记录所有传递依赖版本。直接手动 pip install torch 和通过项目配置安装最终效果一致。使用项目配置的优势是把依赖写进工程配置，便于项目分发。

2.4 练习 4：最小可复现示例 MRE 构建

实验内容

```
mkdir -p ex04
```

创建实验目录

```
cd ex04
```

进入工作目录

```
vim origin_bug.py
```

编写带有冗余代码的bug程序

```
python3 origin_bug.py
```

复现原始bug

```
vim mre.py
```

删减无关代码，得到最小复现脚本

```
vim mre_note.md
```

记录删除的代码以及删除理由

```
python3 mre.py
```

验证最小脚本仍然可以复现bug

实验结果

最小可复现示例MRE只保留触发bug必不可少的代码，删除所有无关变量、打印、分支逻辑。提交Issue时附上MRE，维护者不需要阅读大量无关代码，可以快速定位故障根源。删减过程中要反复运行脚本确认bug没有消失。mre_note.md记录删减内容，用于说明哪些逻辑和bug无关。

2.5 练习 5: Issue 信息完整性评估

实验内容

```
mkdir -p ex05
创建实验目录
cd ex05
进入工作目录
vim raw_issue.txt
复制粘贴原始issue文本
vim issue_analyse.md
评估issue信息完整程度
cat issue_analyse.md
查看评估文档
```

实验结果高质量 issue 需要写明运行环境、复现步骤、实际输出、期望输出。如果缺少关键信息，维护者无法复现 bug，只能来回追问提交者。大量沟通会消耗维护者时间，降低项目处理问题的效率。评估时重点检查：复现命令、版本信息、报错堆栈是否齐全。信息残缺的 issue 需要补充上述内容之后才便于处理。

2.6 练习 6: PyTorch 代码修复与测试验证

实验内容

```
mkdir -p ex06
创建实验目录
cd ex06
进入工作目录
vim bug_torch.py
写入存在bug的pytorch代码
```

```
vim ai_log.md
记录AI修复代码与生成的单元测试
pip install torch pytest
安装pytorch与测试框架
vim test_torch.py
复制AI输出的单元测试
pytest test_torch.py
执行测试验证修复效果
```

实验结果

拿到AI修复代码不能直接信任，必须落地运行单元测试。
单元测试可以确认bug被真正修复，避免出现伪修复。
ai_log.md保存交互记录，方便追溯智能体给出的改动。
PyTorch代码容易出现张量维度、设备不匹配类bug。
测试用例可以复现这类问题，保证修改之后行为符合预期。

2.7 练习 7：智能体开发约束与可编辑安装

实验内容

```
mkdir -p ex07
创建实验目录
cd ex07
进入工作目录
mkdir agent_demo
创建简单包目录
vim agent_demo/__init__.py
编写待修改的业务代码
vim AGENTS.md
编写智能体约束指导文档
vim agent_log.md
记录AI提示、改动、测试结果
pip install -e .
可编辑模式安装项目
python3 -c "import agent_demo; print(agent_demo.run())"
运行代码验证结果
```

实验结果

AGENTS.md给编程智能体提供项目上下文与硬性约束。

写明允许修改的文件、编码约束、测试命令，减少AI乱改无关文件。

agent_log.md留存实验记录，方便追溯智能体输出。

智能体开发场景，约束文档适合大型项目；

简单小任务可以直接使用提示词，不需要独立约束文件。

即使有约束文档，人工依然需要校验运行结果。

2.8 练习 8：推理模块资源文件打包

实验内容

```
mkdir -p ex08
```

创建实验目录

```
cd ex08
```

进入工作目录

```
mkdir infer_mod
```

创建推理模块目录

```
vim infer_mod/__init__.py
```

编写pytorch推理代码

```
touch infer_mod/model.bin
```

模拟模型权重资源文件

```
vim pyproject.toml
```

配置包，设置包含资源文件

```
python3 -m build
```

构建sdist源码分发包

```
mkdir unpack_sdist
```

创建解压目录

```
tar -xf dist/*.tar.gz -C unpack_sdist
```

解压源码包

```
ls unpack_sdist/*/infer_mod
```

检查打包出来的文件

实验结果默认打包只会包含‘py’源码，二进制权重、配置文件会被忽略。package-data 配置项声明需要随包发布的非 python 资源文件。构建 sdist 之后解压查看，可以确认资源是

否被打进分发包。PyTorch 项目发布模型权重时，必须显式配置资源包含规则。否则用户安装包之后会缺失权重文件，推理代码运行失败。

2.9 练习 9：开源项目提交信息重写

实验内容

```
mkdir -p ex09
创建实验目录
cd ex09
进入工作目录
git clone https://github.com/curl/curl.git
克隆curl开源项目
cd curl
进入项目源码目录
git log --oneline -20
浏览最近20条提交记录
git show 提交哈希值
查看某次提交diff与提交说明
cd ..
回到外层目录
vim commit_rewrite.md
记录原始劣质提交，重写规范提交信息
cat commit_rewrite.md
查看改写结果
```

实验结果劣质提交信息只写“修复 bug”这类简短描述，无法说明改动动机。规范提交分为三段：问题描述、解决方案、改动带来的影响。便于团队成员阅读版本历史，快速理解修改背后的业务背景。事后补写提交背景信息十分麻烦，开发时就写好提交说明价值很高。git show 可以同时查看提交说明和代码 diff，是版本排查重要工具。

2.10 练习 10：代码注释搜索与设计意图分析

实验内容

```
mkdir -p ex10
创建实验目录
```

```
cd ex10
进入工作目录
git clone https://github.com/curl/curl.git
克隆curl源码
cd curl
进入源码目录
grep -r "TODO" --include="*.c" .
搜索TODO注释
grep -r "why not" --include="*.c" .
搜索why-not类注释
cd ..
回到外层目录
vim comment_analysis.md
记录找到的注释，分析缺失带来的损失
cat comment_analysis.md
查看分析文档
```

实验结果

TODO注释标记未来需要修复、补充的工作。

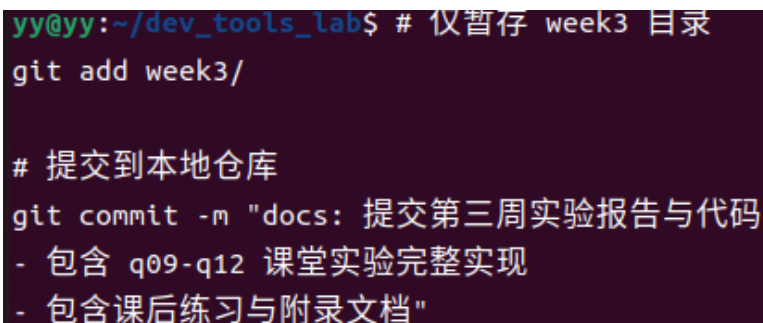
why-not注释解释为什么不采用某种更直观的写法。

外部文档引用可以快速跳转查阅设计文档。

血泪教训注释记录曾经踩过的坑，防止后人重复犯错。

缺少这类注释，后续维护者只能阅读代码猜测设计者意图。

会增加理解成本，很容易重复踩历史已经遇到过的bug。



```
yy@yy:~/dev_tools_lab$ # 仅暂存 week3 目录
git add week3/

# 提交到本地仓库
git commit -m "docs: 提交第三周实验报告与代码
- 包含 q09-q12 课堂实验完整实现
- 包含课后练习与附录文档"
```

图 13: Enter Caption

GitHub 链接: <https://github.com/123yy-cell/-.git>